CSE 144 Final Project Report Transfer Learning Challenge

Member 1: Suchana Poudel

CruzID: 2166770

Spring 2026

1 Introduction

1. **Problem goal and setting**

   This project trains a 100 class image classifier on a dataset sampled from multiple image sources, with approximately 10 training images per class. The model is evaluated on 1036 unlabeled test images.

2. **Why transfer learning is appropriate.**

   With only 10 images per class, there is not enough data to train a neural network from scratch. Instead, we use transfer learning starting from a model already trained on millions of images and fine-tuning it for our specific 100 classes. This way the model already knows how to recognize general image features, and we just teach it our specific categories.

3. **Brief summary of approach and main result**

   Fine-tuned a Swin Transformer Base model pretrained on ImageNet-22k using a two-phase training strategy, first training only the classifier head, then fine-tuning all layers at a lower learning rate. Combined with strong data augmentation, label smoothing, and TTA achieved a Kaggle public leaderboard score of 87%.

2 Dataset

1. Number of classes and dataset sizes

Split	**Images**	**Classes**	**Images per class**
Train	**1036**	**100**	**10**
Test	**1036**	**-**	**-**
Validation	**0**	**-**	**Using a full set(for the final)**

2. **Directory structure and label mapping.**

   The train folder contains 100 subfolders named 0 to 99, each with 10 images. The test folder contains 1036 unlabeled images. A critical issue is that Python's default alphabetical sorting orders folders as 0, 1, 10, 11, 12..instead of 0, 1, 2, 3... which scrambled the labels. We fixed this by sorting folders numerically so that folder "0" maps to label 0, folder "1" maps to label 1, and so on up to folder "99" maps to label 99.

3. **Preprocessing and data augmentation.**

   Training: As we learned in lecture

- Resize to 256×256, standardizes all images to the same size
- RandomHorizontalFlip, mirrors the image horizontally
- RandomRotation(20°), rotates image to simulate different angles
- ColorJitter (brightness=0.4, contrast=0.4, saturation=0.3, hue=0.1)
- RandomResizedCrop(224), crops a random region, simulates different zoom levels
- RandomGrayscale(p=0.1), occasionally removes color, improves robustness
- Normalize with ImageNet mean and std

Inference:

- Resize to 224×224
- Normalize with ImageNet mean and std

3 Implementation

3.1 Model

1. **Pretrained backbone.** We used swin_base_patch4_window7_224.ms_in22k_ft_in1k from the timm library, a Swin Transformer Base pretrained on ImageNet-22k (22 million images, 22,000 classes) then fine-tuned on ImageNet-1k. This model was chosen because:
   a. ImageNet-22k pretraining provided much richer features than ImageNet-1k
   b. The model generalizes well to small datasets due to strong pretrained representations
2. **Architecture changes**

   The original 1000-class classification head was replaced with a linear layer outputting 100 classes: timm.create_model('swin_base_patch4_window7_224.ms_in22k_ft_in1k', pretrained=True, num_classes=100)

3. **Fine-tuning strategy.**

   I used two-phase training as taught in lecture:

   a. Phase 1:

      - Freeze all backbone layers, train only the new classification head for 10 epochs at lr=1e-3. This warms up the head before touching pretrained weights.

   b. Phase 2:

      - Unfreeze all layers, fine-tune the entire network for 30 epochs at lr=1e-4 with CosineAnnealingLR.

3.2 Training

1. **Loss function and optimizer.**
   a. Loss: CrossEntropyLoss with label_smoothing=0.1 to reduce overconfident predictions
   b. Phase 1: Adam optimizer, lr=1e-3
   c. Phase 2: AdamW optimizer, lr=1e-4, weight_decay=0.01
2. **Hyperparameters.**
   a. Batch size: 32
   b. Phase 1: 10 epochs, CosineAnnealingLR (T_max=10)
   c. Phase 2: 30 epochs, CosineAnnealingLR (T_max=30)
   d. Random seed: 42
3. **Hardware and software.**
   a. Hardware: Apple MacBook Air M4 (MPS backend)
   b. PyTorch: 2.8.0, torchvision: 0.23.0, timm: latest, Python 3.9

4 Experiments

1. **Baseline setup.**

   Started with EfficientNet-B2 pretrained on ImageNet-1k, fine-tuned for 20 epochs using an 80/20 train/val split. This reached 63.3% local validation accuracy and was used as our development baseline. I did not submit to Kaggle. I was waiting to hit higher in local.

2. **Hyperparameter tuning and validation method.**

   During development I used an 80/20 train/val split to monitor overfitting. For final submissions we trained on the full dataset, I did not do value split to maximize training data, using the Kaggle public leaderboard as our evaluation metric.

Experiments conducted:

- Tested learning rates: 1e-3, 5e-5, 1e-4, found 1e-4 optimal for Phase 2
- Tested epoch counts: 20, 30, 50, found 30 optimal
- Lower LR of 5e-5 with 50 epochs decreased Kaggle accuracy from 72% to 70%

3. **Ablations.**

Model	Setting	Result
EfficientNet-B2	SEED=42, 20 epochs, original code, label bug present	25% val (scrambled labels)
EfficientNet-B2	SEED=42, 20 epochs, label bug fixed	58.1% val
EfficientNet-B2	SEED=42, 20 epochs, stronger augmentation	58.6% val
EfficientNet-B2	SEED=123, 30 epochs	59.1% val
EfficientNet-B2	SEED=42, 30 epochs, label smoothing, stronger augmentation	63.3% val
EfficientNet-B2	SEED=42, 50 epochs, lr=5e-5	59.1% val
EfficientNet-B4	SEED=42, 30 epochs, lr=1e-4, all data, TTA	Phase 2: 97.4% 72% Kaggle
EfficientNet-B4	SEED=42, 50 epochs, lr=5e-5, all data, TTA	Phase 2: 96.4% 70% Kaggle
Swin Large	ImageNet-22k, 30 epochs	Stopped, too large
Swin Base	ImageNet-22k, SEED=42, 30 epochs, lr=1e-4, all data, TTA	Phase train acc: 99% 87% Kaggle

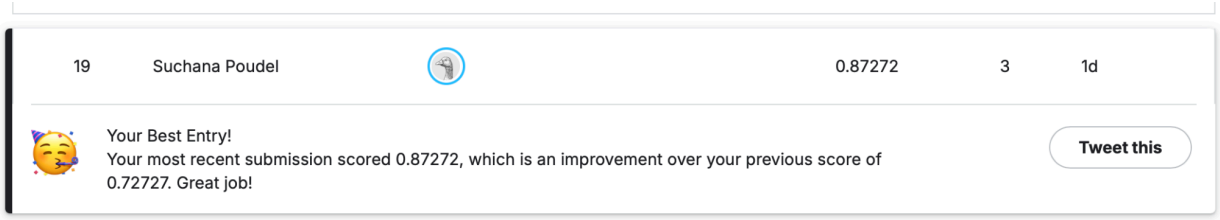
5. Results

1. **Training and validation accuracy.**

Model	Phase 1	Phase 2	Result	Kaggle
EfficientNet-B2	25%	25	25% v	Not submitted
EfficientNet-B2	42%	58.1	58.1% val	Not submitted
EfficientNet-B2	42%	58.6	58.6% val	Not submitted
EfficientNet-B2	42%	63.3	59.1% val	Not submitted
EfficientNet-B2	42%	63.3	63.3% val	Not submitted
EfficientNet-B2	42%	59.1	59.1% val	Not submitted
EfficientNet-B4	64.6	97.4	none	72% Kaggle
EfficientNet-B4	64.6	96.4	none	70% Kaggle
Swin Large	Stopped	Stopped	Stopped	Stopped
Swin Base	84.2	99%	none	87% Kaggle

Kaggle public leaderboard score.

- Best submission: Swin Base at 87%
- Second best: EfficientNet-B4 at 72%



Key observations.

- The label ordering bug capped accuracy at 25%, fixing it immediately improved to 58%
- Each model improvement brought clear gains:

B2 -63.3% val, B4 - 72% Kaggle, Swin Base -87% Kaggle

- Switching to ImageNet-22k pretraining was the single biggest jump (+15% over B4)
- Gap between Swin Base train accuracy (99%+) and Kaggle score (87%) reflects expected overfitting on 10 images per class.
- TTA added almost 1-2% with submissions.

6. Discussion

1. **What worked best and why.**
    - Switching to Swin Transformer Base with ImageNet-22k pretraining was the biggest improvement, jumping from 72% to 87% on Kaggle. This model was pretrained on 22 million images, giving it much richer features than EfficientNet which was trained on only 1.2 million images.
    - Two-phase training worked well. Freezing the backbone first let the new classification head stabilize before fine-tuning the full network at a lower learning rate.
    - Label smoothing (0.1) helped prevent the model from becoming overconfident on the small training set.
    - Test Time Augmentation added about 1-2% improvement at no training cost by averaging predictions across 5 augmented versions of each test image.
    - Training on all 1036 images instead of keeping a validation split gave the model more data for the final submission.
2. **Failure cases and overfitting observations.**
    - The label ordering bug in the first experiment caused only 25% accuracy because alphabetical sorting of folder names scrambled all class assignments.
    - Swin Base reached 99%+ train accuracy but only 87% on Kaggle, showing overfitting. With only 10 images per class this is expected and hard to avoid.
    - Lowering the learning rate to 5e-5 in Phase 2 hurt performance (70% vs 72%), showing that too slow fine-tuning does not help on small datasets.
    - Swin Large was too large and my computer started lagging and had to be stopped mid-training.
3. **Limitations and next improvements.**
    - Ensemble Swin Base and EfficientNet-B4 predictions together to reduce variance and likely add 2-3%
    - Use larger input resolution (384x384) for Swin to capture more image detail
    - Add Mixup or CutMix augmentation as taught in CSE 144 lecture to reduce overfitting
    - Increase TTA from 5 to 10 augmentations for a small consistent gain
    - Add dropout to the classifier head to directly reduce overfitting

7. Reproducibility

1. Random seeds and determinism settings.
- The following fixed seeds were set at the start of every training run:

SEED = 42 torch.manual_seed(42) random.seed(42) numpy.random.seed(42)

2. Package versions and environment setup.

Install all required packages:

pip3 install torch torchvision timm pandas pillow numpy

Environment used:

- Python: 3.9
- PyTorch: 2.8.0
- torchvision: 0.23.0
- timm: latest
- Hardware: Apple MacBook Air M4 (MPS backend)(What I used) (CUDA GPU can be used)

Exact commands to train and generate submission-

Download the kaggle data:

Place dataset in: Desktop/cse144_project/ucsc-cse-144-spring-2026-final-project/

Run training and inference: python3 train.py

This will:

- Train Swin Base on all 1036 training images
- Save model weights to pretrained.pth
- Create submission_swin.csv automatically

Download model weights from Google Drive:

Step 5 - To load model weights without retraining:

model = timm.create_model('swin_base_patch4_window7_224.ms_in22k_ft_in1k', num_classes=100) model.load_state_dict(torch.load('pretrained.pth'))

Place weights file at: ucsc-cse-144-spring-2026-final-project/pretrained.pth

https://drive.google.com/file/d/14F4VKc-excqh0xyQSlIO4O6H-yRzSZOm/view?usp=share_link

8.Team

-Solo project

Suchana Poudel

9. References

1. TorchVision documentation: https://pytorch.org/vision/stable/models.html
2. PyTorch documentation: https://pytorch.org/docs/stable/index.html
3. For the pretrained-timm library (PyTorch Image Models): https://github.com/huggingface/pytorch-image-models
4. Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., and Guo, B. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. ICCV 2021. https://arxiv.org/abs/2103.14030
5. Tan, M. and Le, Q. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML 2019. https://arxiv.org/abs/1905.11946